A PROJECT REPORT ON

AUTOMATIC SPEECH RECOGNITION FOR INDIC LANGUAGES

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Manish Godbole	Exam No: B400050403
Kaustubh Joshi	Exam No: B400050428
Aditya Kadu	Exam No: B400050431

DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025

CERTIFICATE

This is to certify that the project report entitled

AUTOMATIC SPEECH RECOGNITION FOR INDIC LANGUAGES

Submitted by

Manish Godbole	Exam No: B400050403
Kaustubh Joshi	Exam No: B400050428
Aditya Kadu	Exam No: B400050431

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Dr. Mukta Takalikar** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Dr. Mukta Takalikar
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place:Pune
Date:24/04/2025

ACKNOWLEDGEMENT

It gives us great pleasure in presenting the project report on ***'AUTOMATIC SPEECH RECOGNITION FOR INDIC LANGUAGES'****.*

We are truly grateful to ***Dr. Mukta Takalikar*** *for all the help and guidance provided. Their kind support and valuable suggestions were incredibly helpful throughout.*

We are also grateful to ***Dr. G. V. Kale****, Head of Computer Engineering Department and Principal* ***Dr. Sanjay Gandhe*** *for their encouragement and support.*

In the end our special thanks to ***Mr. Harshad Saykhedkar****, Lightbees Technologies Pvt. Ltd. We would also like to thank all faculty members for their complete cooperation to complete this report.*

Manish Godbole
Kaustubh Joshi
Aditya Kadu
(B.E. Computer Engg.)

ABSTRACT

In collaboration with Lightbees, a fintech start-up, this final year B.E. project focuses on enhancing AbleCredit, a flagship product aimed at simplifying loan applications for MSME. The project targets the bottlenecks in traditional credit processes, particularly the challenges faced by small and medium businesses in accessing formal financial systems. By analyzing customer needs and pain points, a user-centric design approach was adopted to maximize accessibility and functionality.

The system leverages Artificial Intelligence (AI), Machine Learning (ML), Natural Language Processing (NLP), and Digital Signal Processing (DSP) to build intelligent modules that assist in document verification, voice-based form filling, and sentiment analysis. These technologies contribute toward streamlining the loan application process by reducing manual intervention, ensuring higher accuracy, and enabling quicker decision-making through automated checks and risk profiling.

Overall, this project embodies the integration of theoretical knowledge with practical application, addressing real-world challenges in financial technology. The proposed solutions aim not only to enhance the usability and efficiency of AbleCredit but also to contribute meaningfully to the broader fintech landscape by offering scalable, intelligent, and inclusive approaches to digital lending.

Contents

List of Figures

CHAPTER 1
INTRODUCTION

1.1 Overview

The rapid evolution of technology in the financial sector has brought about innovative solutions to streamline processes and enhance accessibility. In this context, our B.E. project, sponsored by Lightbees, a fintech startup known for their product *AbleCredit.com*, aims to develop and refine an Automatic Speech Recognition (ASR) system for Indic languages. The goal of this project is to create a speech-based interface that facilitates easy loan acquisition for Micro, Small, and Medium Enterprises (MSME), providing them with a faster and more user-friendly experience in accessing financial services.

India, with its diverse linguistic landscape, presents unique challenges for ASR systems. Existing solutions often struggle with regional languages, dialects, and accents, limiting their adoption across the country. Our project aims to bridge this gap by designing an ASR system tailored specifically for Indic languages like Hindi, Marathi, Kannada, and others. The system will integrate Natural Language Processing (NLP) techniques to ensure accurate transcription and interpretation of spoken inputs, allowing users to seamlessly apply for loans through the *AbleCredit.com* platform.

In addition to developing the ASR system, we created a mobile application to serve as the interface between users and the fintech platform. This application incorporates voice commands and provides real-time feedback, ensuring a smooth interaction for MSMEs who may have limited literacy or access to traditional digital services. By leveraging speech technology, this project seeks to empower underserved communities and improve financial inclusion in India.

The project results in a scalable, efficient solution that addresses the linguistic and technological barriers faced by MSMEs, ultimately contributing to a more inclusive digital financial ecosystem.

1.2 Motivation

India's population is incredibly linguistically diverse, yet many digital services remain accessible only to speakers of English or a handful of regional languages. This creates a digital divide that excludes a significant portion of the population, especially small-scale business owners. We were motivated to build an ASR solution that could bridge this gap and provide voice-driven financial access.

1.3 Problem Definition and Objectives

Problem: Existing ASR systems are not optimized for Indic languages, especially in noisy environments or rural dialects. Additionally, integration into low-resource mobile apps is challenging due to size and performance constraints.

Objectives:

- Develop a lightweight ASR model for Indic languages.
- Integrate the ASR model into a Kotlin-based Android app.
- Enable seamless voice-based loan application processes for MSMEs.
- Enhance accessibility for users with limited literacy or digital experience.

1.4 Project Scope & Limitations

The project focuses on the development of a functional speech recognition system for a few select Indic languages (initially Hindi and Marathi). Due to hardware and time limitations, full coverage of all Indic languages and dialects was not feasible. The mobile app also targets Android platforms only.

1.5 Methodologies of Problem Solving

We used OpenAI's Whisper ASR model as our base due to its multilingual capabilities. However, Whisper is resource-heavy in its raw format and not suitable for direct mobile deployment.

- **Model Size and Optimization:** Whisper models are large and computationally expensive. We had to convert them to a lighter format using **Whisper.cpp**, which allows for faster inference and better resource management.
- **GGML Conversion:** We converted the Whisper model to the **GGML** (Grokking General Machine Learning) format, which significantly reduces size and supports faster inference, making it suitable for edge devices.

- **Kotlin Integration:** Integrating the optimized ASR model into a Kotlin-based Android app was non-trivial. We used the NDK (Native Development Kit) to call the C++ inference engine from within Kotlin, and had to manage JNI bindings carefully to ensure real-time responsiveness.
- **Latency and Accuracy:** Achieving low latency while maintaining transcription accuracy was a challenge, especially when dealing with varied regional accents and background noise. We used post-processing techniques and lightweight NLP filters to refine the final transcriptions.
- **UI/UX for Low-Literacy Users:** We designed a simple voice-first interface with minimal text, visual cues, and instant feedback to ensure usability for non-tech-savvy users.

The final solution achieved a good balance between accuracy, performance, and mobile readiness, marking a step forward in inclusive fintech application development.

CHAPTER 2
LITERATURE SURVEY

Literature Review

Liddy, E.D

Natural Language Processing (NLP) is a process of computer-assisted text analysis that has theoretical and practical foundations. Since it is an expanding area of research and development there is no clear definition. Nevertheless, there are some features that the definition would have to contain.

Diksha Khurana, Aditya Koli, Kiran Khatter, Sukhdev Singh

Natural Language Processing (NLP) is a field that has gained significant attention recently for its ability to computationally represent and analyze human language. Its applications have now spread to different areas like machine translation, spam detection of emails, information extraction, summarization, healthcare and answering questions, to name a few. In this paper, the authors start by noting four phases through a discussion of different levels of NLP and components of Natural Language Generation, followed by the historical overview and the evolution of NLP. They then cover the current state of the art, which includes the different applications of NLP, the new trends that are emerging, and the challenges that already exist. In the last part, they summarize what is available in the datasets, models, and evaluation metrics of NLP.

Aditya Jain, Gandhar Kulkarni, Vraj Shah

For instance, modern text processing algorithms assign entities to categories and the preferences of users establish them. These algorithms are present in features like smart replies and smart suggestions that can be used in different applications, which are designed to reduce the workload of users and the time spent in providing by accurate and efficient responses. Despite the significant developments in the field over the past decade, the task of handling speech processing issues yet to be finished. Neural networks and deep learning techniques play a significant part in the issues of both the text and the speech process for more efficient industrialized activities. Thus, these innovations bring the accuracy level of results near the level of human comprehension. AI systems execute text and speech processing algorithms for evaluating the user needs mainly according to input classification which is the cause of more individualized results. In fact, AI systems combine diverse

text and speech processing algorithms, and they move still higher in the levels of accuracy.

M. I. Jordan, T. M. Mitchell

Machine learning is a subset of computer science that covers two main areas: How can we develop computer systems that learn and get better on their own through experience? What are the statistical, computational, and information-theoretic principles that underlie all the learning systems, whether computers, humans, or organizations? Machine learning is not only complex research but also an optimization problem while in practice, it has enabled the application of multiple software, which has been implemented in various areas.

Qifang Bi, Katherine E. Goodman, Joshua Kaminsky, Justin Lessler

Artificial intelligence (AI) focuses on tasks that are achievable through machines, such as forecasting and optimization. The core principle behind AI systems is improving performance by minimizing errors when executing specific tasks. This process is often conceptualized as the machine exploring numerous alternative scenarios to achieve optimal solutions. While the distinction between machine learning and statistical methods can seem subtle, particularly in practical applications, there is a clear differentiation in their development and technical implications. Machine learning, though it may utilize statistical techniques like LASSO or stepwise regression, emphasizes the continuous improvement of algorithms through data-driven learning. In contrast, traditional statistical methods often prioritize hypothesis testing and model validation. These distinctions reflect the different goals and methodologies inherent in machine learning versus statistics, despite their shared reliance on data analysis.

Batta Mahesh

Machine learning (ML) basically is the branch of science that deals with development, implementation, and examination of algorithms and statistical models allowing computer systems to perform given tasks without human input. Learning algorithms are integrated into most of the applications we use every single day. Say a web search engine such as Google, for example, is one of the major reasons that it runs so well and provides such a good service is thanks to a learning algorithm that has learned through a large

dataset of web pages to rank them in an efficient and accurate way. These algorithms cross different domains such as data mining, image processing, and predictive analytics, among others. The major advantage of the machine learning method is that, when the algorithm has learned how to deal with the data, then it can work out the task without human intervention. This article is an overview of the wide use of machine learning algorithms.

Ian Goodfellow, Yoshua Bengio, Aaron Courville

"Deep Learning" refers to a comprehensive review of the latest deep learning developments and upcoming research directions. Authors, Ian Goodfellow, together with his Ph.D. adviser Yoshua Bengio, and Aaron Courville, are the recognized authorities in the field of artificial intelligence (AI).

Yann LeCun, Yoshua Bengio, Geoffrey Hinton

Deep learning is a class of machine learning algorithms that, uses a cascade of multiple layers of nonlinear processing units for feature extraction and transformation. Such methods have enabled major advances in fields like speech recognition, visual object recognition, and object detection; they have also played important roles in substantial improvements in machine learning technologies more generally, producing large performance gains across diverse applications such as drug discovery, genomics, artificial intelligence (AI), materials science. Deep learning operates on complex patterns in large amounts of data, allowing the neural network to adjust its internal parameters — which such as weights and biases are also known as — that dictate how computations for each layer's representation are determined from those in preceding layers (backpropagation algorithm). While deep convolutional nets have led to major advances in a variety of tasks, recurrent networks excel in other areas such as sequential data like text and speech.

Yanming Guo, Yu Liu, Ard Oerlemans, Songyang Lao, Song Wu, Michael S. Lew

Deep learning: a machine learning method based on learning data representations, as opposed to task-specific algorithms. It is being constantly improved upon and has been heavily utilized in many traditional AI areas such as semantic parsing, transfer learning , NLP, computer vision etc. So how could the rather lackluster results of early deep learning research, and researchers' limited attention to this field in general be reconciled with its quick growth today? Restrospectively speaking, three things were key: massively

increased computation power like GPUs, which everybody uses for neural networks now; cheaper commodity computing hardware; and improvements in algorithms.

Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, Ilya Sutskever

The work examines the potential of speech processing systems that have been trained on vast numbers of audio transcripts from sources found in online databases. When scaled up to 680,000 hours of multilingual and multimodal task training, the resulting models shows excellent generalization on standard benchmarks — often matching or surpassing previous fully supervised baselines without any fine-tuning or even any labeled data. Our released models and inference code are meant to serve as a reference point for future studies in robust speech processing.

Javed, T., Doddapaneni, S., Raman, A., Bhogale, K. S., Ramesh, G., Kunchukuttan, A., Kumar, P., Khapra, M. M.

A way of building ASR systems for low-resource Indian languages is addressed through collection and usage of 17,000 hours of speech data across 40 languages from the most varied of domains. To do this, the authors pre- train multiple language models inspired by wav2vec, focusing the pre-training on features such as shared phonemes and languagefamily discriminative representations. All these models are fine-tuned for ASR with nine languages and achieve state-of-the-art performance for three public benchmarks, namely, Sinhala, Nepali, and other low-resource languages, indicating multilingual pretraining works very well in ASR across all the languages.

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Introduction

This chapter describes the essential hardware and software requirements for the development and deployment of the BhaashaTranscribe mobile application. The system integrates an offline speech recognition engine using a GGML-optimized Whisper model, with a Kotlin-based Android interface.

3.2 Hardware Requirements

Table 3.1: Hardware Requirements

Component	**Specification**
Mobile Device	Android smartphone with minimum 4 GB RAM, 2 GHz quad-core CPU, and 1 GB free storage
Training Hardware	CUDA-enabled GPU with at least 12 GB VRAM

3.3 Software Requirements

Table 3.2: Software Requirements

Software	**Details**
Operating System	Android 9.0 (Pie) or higher
Development Environment	Android Studio (latest stable release)
Programming Language	Kotlin with Jetpack Compose and Android SDK
Speech Model Frameworks	PyTorch, Whisper.cpp
Model Optimization	GGML quantization (8-bit)
Audio Processing Tools	FFmpeg, SoX
GPU Framework	CUDA Toolkit (compatible with 12 GB GPU VRAM)

3.4 Dependencies and Libraries

- `PyTorch` — for training and fine-tuning the Whisper ASR model.
- `Whisper.cpp` — for GGML conversion and lightweight inference.
- `Kotlin + Jetpack Libraries` — for mobile application development.
- `FFmpeg / SoX` — for audio format conversion and preprocessing.
- `NumPy, TorchAudio` — used in preprocessing and data loading stages.

3.5 Assumptions and Dependencies

In this section, we study the assumptions and dependencies related to Bhaasha-Transcribe and its development.

- The mobile device has a minimum of 4 GB RAM and a multi-core CPU.
- Users are familiar with using mobile apps and can speak Hindi, Marathi, or a mix of both.
- The application will be used in semi-urban and rural areas, where internet access may not be consistent.
- All processing is assumed to be done offline; no server-side speech recognition is used.
- The application depends on Whisper.cpp, a lightweight inference engine, and GGML-quantized models.
- The training environment requires CUDA-enabled GPUs with at least 12 GB VRAM.

3.6 Functional Requirements

This section details the core functionalities that BhaashaTranscribe must support.

3.6.1 System Feature 1: Speech-to-Text Transcription

- The system shall capture audio from the user's microphone.
- The system shall transcribe spoken content into written text in real time.
- The system shall work with Hindi, Marathi, and code-switched (Hindi-Marathi-English) speech.

3.6.2 System Feature 2: Offline Execution

- The transcription must function entirely offline.
- No data should be sent to external servers or require cloud infrastructure.
- The system shall preserve user privacy by storing or processing all data locally.

3.6.3 System Feature 3: Audio Processing and Noise Handling

- The system shall preprocess audio using MFCC/STFT to enhance transcription accuracy.
- It should handle background noise to a reasonable extent using pretrained filters.

3.6.4 System Feature 4: Result Display and Storage

- The transcribed text shall be displayed to the user in the app interface.
- The transcription result may be saved locally in a file or copied to clipboard.

3.6.5 System Feature 5: User Interface and Language Setting

- The application shall provide a clean UI using Jetpack Compose.
- Users can select preferred language or let the app auto-detect based on input.

3.7 External Interface Requirements

3.7.1 User Interfaces

- The user interacts with the system through a mobile application developed in Kotlin using Jetpack Compose.
- The app features a clean and minimal interface with large touch areas for usability in semi-urban and rural environments.
- The interface provides options to:
  - Start and stop audio recording.
  - View real-time transcription results.
  - Select or auto-detect preferred language (Hindi/Marathi).
  - Save or copy the transcribed text.
- The UI supports both left-to-right and Indic-script-friendly rendering for Devanagari text.

3.7.2 Hardware Interfaces

- **Microphone:** Captures real-time audio input from the user.
- **Mobile CPU:** Executes the GGML-quantized Whisper model entirely offline.
- **RAM:** Minimum 4 GB RAM is required to load and run the transcription model smoothly.
- **Storage:** Minimum of 1 GB free space to store the model, intermediate files, and transcriptions.

3.7.3 System Flowchart

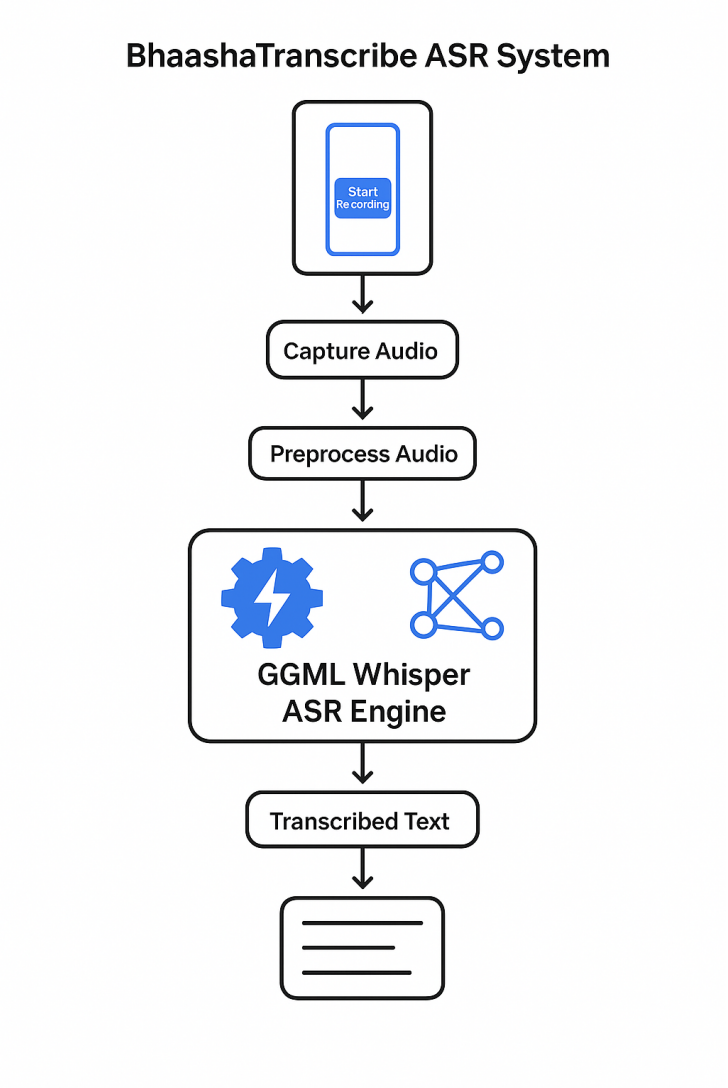

Figure 3.1: System Flowchart of BhaashaTranscribe ASR

3.7.4 Software Interfaces

1. **Operating System:** Development and training environments support Linux, macOS, and Windows.
2. **Android OS:** Required for the deployment of the mobile app (Android 9.0 or higher).
3. **Jupyter Notebook:** Used for pre-processing audio datasets, model training, and experimentation.
4. **Programming Language:** Python (for model training and conversion), Kotlin (for app development).
5. **Whisper.cpp and GGML:** For optimized, quantized speech recognition.

3.7.5 Communication Interfaces

- No external communication is required at runtime, as the application functions completely offline.
- Communication may occur during development (e.g., Git repositories, dataset access) or training (e.g., CUDA libraries, Python packages).
- The mobile app may optionally share saved transcription files via local device sharing tools.

3.8 Nonfunctional Requirements

3.8.1 Performance Requirements

- The app must perform transcription within 5 seconds for a 10-second audio clip on devices with 4 GB+ RAM.
- GGML-optimized models must load in under 2 seconds.
- Background noise should not reduce accuracy by more than 10%.

3.8.2 Safety Requirements

- The app must not crash during recording or transcription, even under memory stress.
- The system must validate audio input formats and safely reject corrupt or unsupported files.

3.8.3 Security Requirements

- All user data is processed locally and not uploaded to the cloud.
- No audio or transcribed text is stored without user consent.
- Android permissions must be limited to only microphone access.

3.8.4 Software Quality Attributes

- **Usability:** Minimalist UI for intuitive use with support for Devanagari script.
- **Reliability:** High uptime, robust error handling, crash resistance.
- **Maintainability:** Modular Kotlin code and well-documented Python training pipeline.
- **Portability:** Works on a variety of Android phones and adaptable for other Indian languages.

3.9 System Requirements

3.9.1 Database Requirements

- No traditional database is used. Transcription logs may be saved as text or JSON files on device storage.
- During model training, datasets are stored in WAV or MP3 format with transcript labels in CSV/JSON.

3.9.2 Software Requirements (Platform Choice)

- Python 3.10 with libraries: PyTorch, NumPy, TorchAudio.
- FFmpeg and SoX for audio preprocessing.
- Whisper.cpp for inference with GGML quantization.
- Android Studio for Kotlin app development.

3.9.3 Hardware Requirements

- CUDA-enabled GPU with at least 12 GB VRAM (for model training and optimization).
- Android device with minimum 4 GB RAM and ARM CPU (for app execution).
- Minimum 1 GB free space for storing models and transcriptions.

3.10 Analysis Models: SDLC Model to be Applied

- The project follows an **Iterative Development Model**, allowing rapid prototyping and improvements.
- Key phases include:
  - Requirement analysis
  - Model training and benchmarking
  - App integration and testing
  - User feedback and optimization
- This model was chosen due to the evolving nature of ASR accuracy and usability testing in real-world conditions.

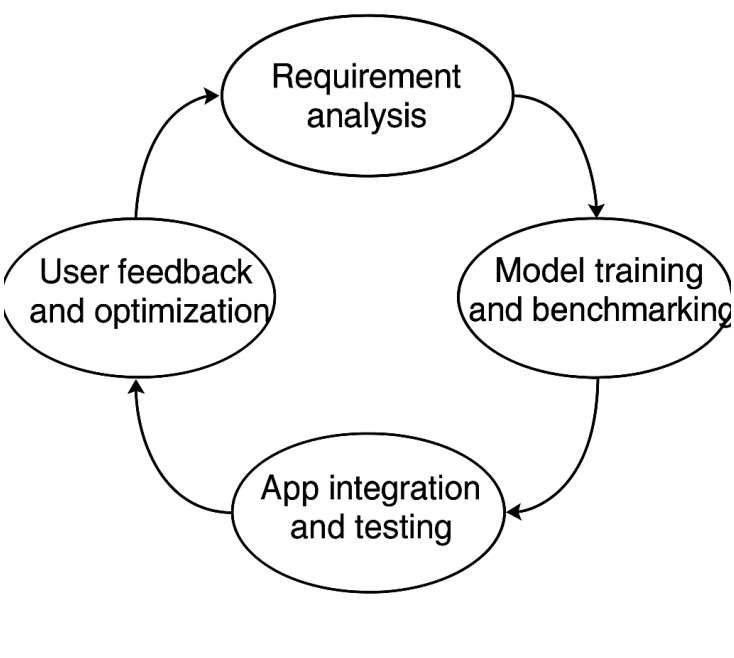

Figure 3.2: SDLC Model: Iterative Approach Used in Project

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

The architecture integrates Whisper.cpp with a Kotlin-based Android application for offline speech recognition. Audio is recorded, processed offline with a GGML-quantized Whisper model, and output is displayed in Devanagari script (Hindi/Marathi).

4.1.1 Mobile Device Layer

- **Audio Recording:** Captures audio using Android's MediaRecorder or AudioRecord API.
- **Audio Processing:** Includes noise reduction and volume normalization.

4.1.2 Offline Model Layer (Whisper.cpp with GGML Quantization)

- **Model Integration:** Whisper.cpp (GGML quantized) is integrated via JNI for offline inference.
- **Model Inference:** Transcribes audio into English text without requiring internet access.

4.1.3 Text Processing Layer

- **Translation:** Converts English text into Hindi/Marathi using local translation models.
- **Devanagari Script:** Outputs the transcription in Devanagari Unicode.
- **Post-Processing:** Corrects transcription errors and formats the output.

4.1.4 User Interface Layer

- **Display Output:** Transcription in Devanagari script is shown on the UI.
- **Features:** Real-time transcription, save/export transcriptions, and text-to-speech.

4.1.5 Performance Optimization

- **Model Optimization:** GGML quantization reduces memory usage and improves inference speed.
- **Energy Consumption:** Optimized to minimize battery drain during processing.

4.1.6 Device-Specific Features

- **Cross-device Compatibility:** Adapts to various Android devices.
- **Adaptive UI:** Adjusts for different screen sizes and resolutions.

4.2 Mathematical Model

Let:

- $x(n)$ be the raw input audio.
- $X(m, k)$ be the STFT output.
- F be the MFCC feature vector extracted from $x(n)$.
- M be the GGML Whisper model.
- T be the transcribed text.

The overall function is:

$$f : x(n) \rightarrow X(m, k) \rightarrow MFCC \rightarrow F \rightarrow M(F) = T$$

Where:

$$X(m, k) = \sum_{n=0}^{N-1} x(n) w(n - mR) e^{-j2\pi kn/N}$$

$$MFCC = DCT(\log(M(|X(m, k)|)))$$

And M is a Transformer model with encoder-decoder layers predicting T using:

$$\mathcal{L}_{CE} = -\sum_{t=1}^{T} y_t \log \hat{y}_t$$

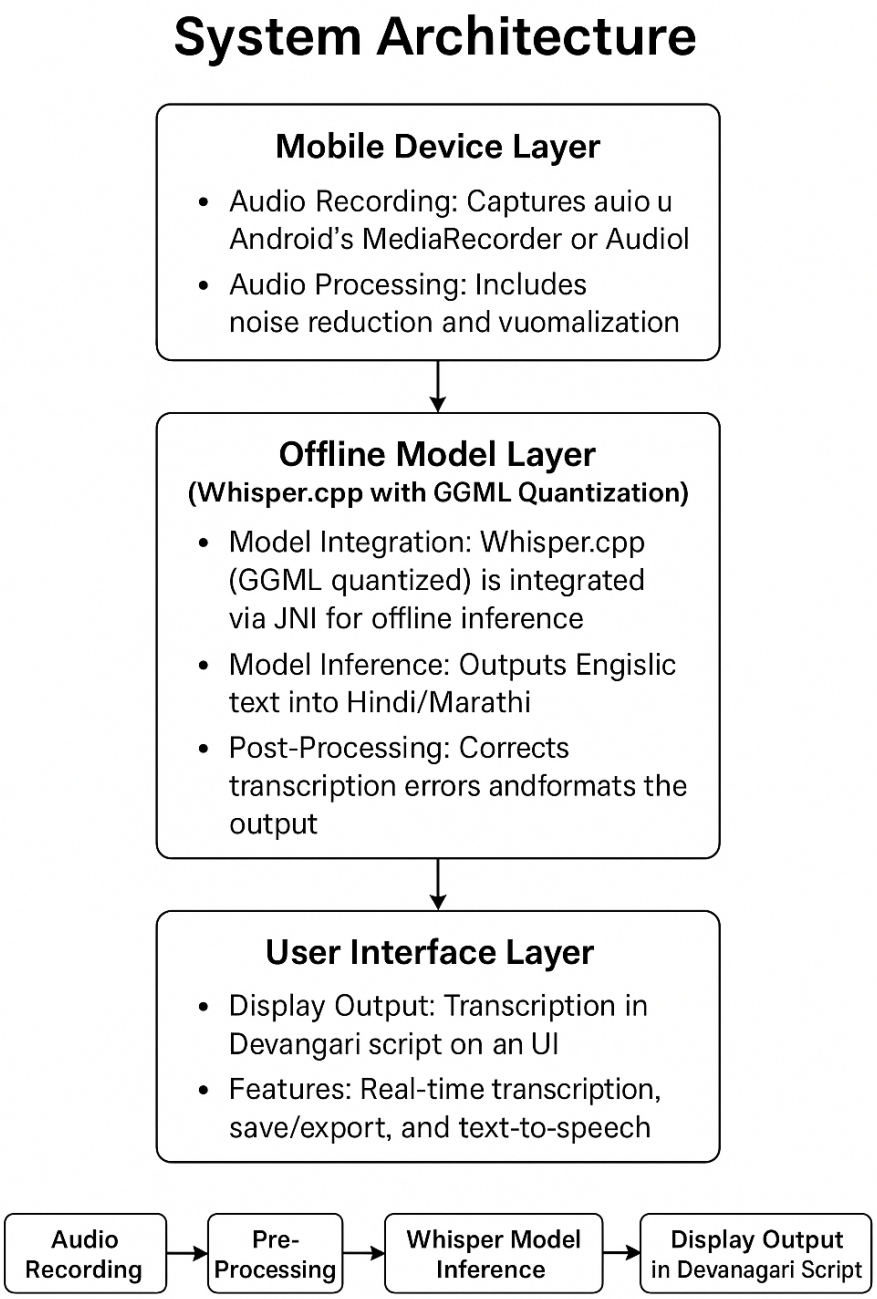

Figure 4.1: System Architecture for Whisper Model Integration with Kotlin-based Android App

4.3 Data Flow Diagram

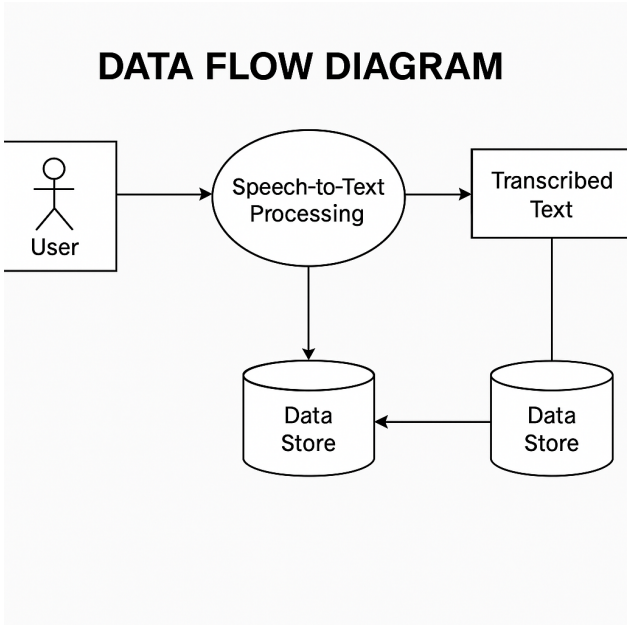

Figure 4.2: Data Flow Diagram

4.4 Entity Relationship Diagram

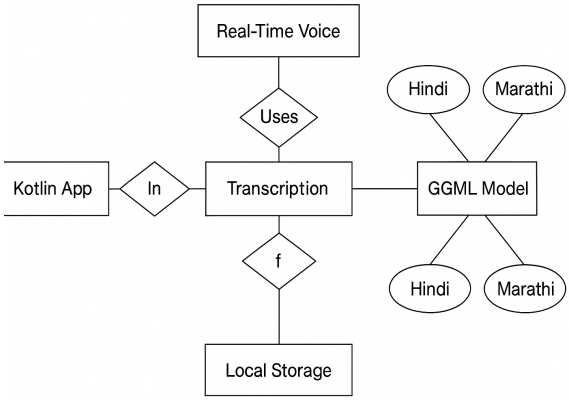

Figure 4.3: Entity Relationship Diagram

CHAPTER 5
PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

- **Project Duration:** August 2024 to April 2025 (9 months)
- **Effort Estimate:** High; involved CUDA-based training, quantization, Kotlin integration, and mobile optimization
- **GPU Usage Cost:** Estimated some thousands of Rupees using AWS GPU instances (e.g., g4dn.xlarge)
- **Model Sizes:** 480 MB (Hindi), 490 MB (Marathi) using 8-bit GGML format for on-device efficiency

5.1.2 Project Resources

- **Human Resources:** 2–3 developers for model training, Kotlin development, and testing
- **Technologies:** Whisper.cpp, PyTorch, CUDA, Kotlin, Android Jetpack
- **Hardware:** AWS GPUs for training, Android mobile devices for testing (mid-range to high-end)
- **Software Tools:** Python, Android Studio, Whisper.cpp, FFmpeg, Git, VS Code

5.2 Risk Management

- Effective risk management was essential to address challenges in speech model deployment on mobile devices

5.2.1 Risk Identification

- **Model Hallucination:** Whisper may produce incorrect transcriptions under noise
- **Accent/Dialect Variability:** Diverse pronunciation may reduce accuracy
- **Mobile Resource Constraints:** Limited RAM/CPU on target devices
- **Privacy/Security Concerns:** Ensuring offline data handling to maintain trust

5.2.2 Risk Analysis

- **High Severity:** Accent variation, device limitations
- **Moderate Severity:** Model hallucination, especially in noisy environments
- **Low Severity:** Network usage risk — mitigated via full offline inference

5.2.3 Overview of Risk Mitigation, Monitoring, Management

- **Mitigation:** Fine-tuning with diverse, augmented datasets and domain-specific speech
- **Monitoring:** Performance logged on various device configurations during testing
- **Management:** Regular qualitative and quantitative evaluation (e.g., WER, CER, latency benchmarks)

5.3 Project Schedule

5.3.1 Project Task Set

- Literature Review and Dataset Collection
- Preprocessing: STFT, MFCC, and audio formatting
- Fine-tuning Whisper models for Hindi and Marathi using CUDA and AWS
- Quantization using GGML and conversion for Whisper.cpp
- Kotlin app development and integration with Whisper.cpp
- UI/UX design using Jetpack Compose
- Evaluation: WER, CER, Latency benchmarks
- Documentation and Report Writing

5.3.2 Task Network

- The task dependencies were assigned as:
  1. Data collection → pre-processing → Model Fine-tuning
  2. Fine-tuning → Quantization → Kotlin Integration
  3. Parallel: UI Design ‖ Mobile Inference Testing
  4. Final: Evaluation → Documentation

5.3.3 Timeline Chart

- The main phases included model training (August-December), mobile integration (January–February), and evaluation/reporting (March-April).

5.4 Team Organization

5.4.1 Team Structure

- The team followed a flat collaborative structure:

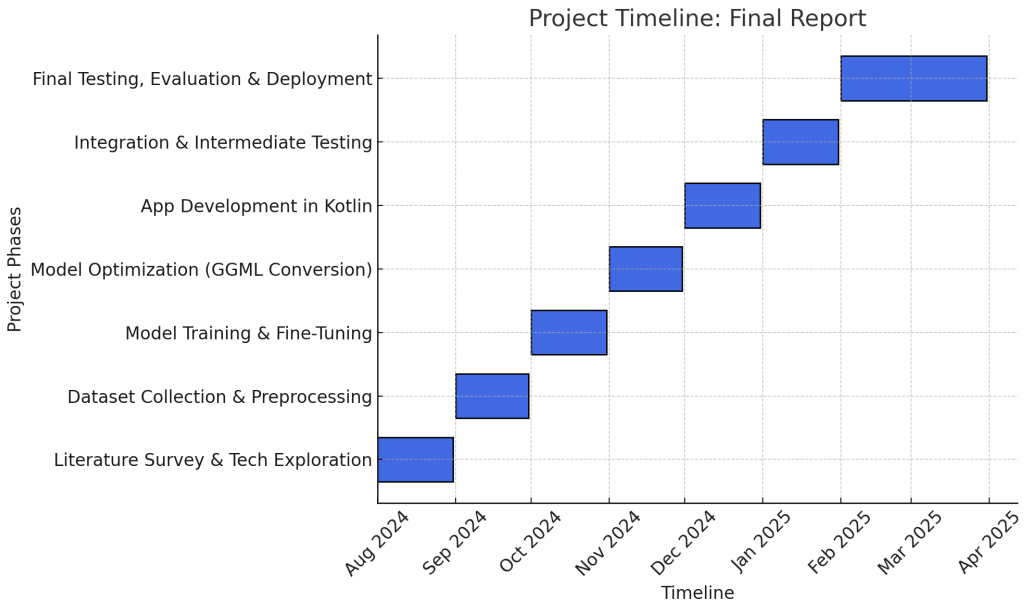

Figure 5.1: Timeline Chart: August 2024 to April 2025

- **Model Development:** All members were responsible for Whisper fine-tuning and GGML quantization
- **Android Development:** Collaborative work on Kotlin, Jetpack, and mobile integration was done.
- **Evaluation and Documentation:** Final testing, benchmarking and reporting was performed.

5.4.2 Management Reporting and Communication

- Weekly sync meetings were held for progress tracking and troubleshooting
- Work was divided via a shared with Documented results.
- Code and models were version-controlled using Git and GitHub
- Final documentation was collaboratively written and reviewed.

CHAPTER 6
PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

- **Data Preprocessing Module:** Loads and prepares the Marathi audio dataset for training.
- **Model Fine-tuning Module:** Trains and evaluates the Whisper model on the Marathi dataset.
- **Quantization Module:** Converts the trained model to a lightweight GGML format using whisper.cpp.
- **Mobile Integration Module:** Embeds the quantized model in a Kotlin-based offline Android application.
- **Evaluation Module:** Computes metrics like WER (Word Error Rate), latency, and performance.

6.2 Tools and Technologies Used

- **Programming Languages:** Python, Kotlin
- **Frameworks and Libraries:** Hugging Face Transformers, Whisper.cpp, Jetpack Compose, PyTorch
- **Data:** Common Voice Marathi Dataset
- **Cloud Platform:** AWS EC2 with GPU
- **Development Tools:** Android Studio, GitHub

6.3 Algorithm Details

6.3.1 Fine-tuning Whisper Model for Indic Languages

Algorithm 1 Model Fine-tuning Procedure

1: Load Common Voice Hindi/Marathi datasets
2: Standardize audio format: 16kHz mono PCM
3: Initialize Whisper feature extractor:
- MFCC filtering
- Log-Mel spectrogram conversion

4: Tokenize text using Whisper's multilingual tokenizer
5: Create sequence-to-sequence mapping:
- Audio features $\rightarrow$ tokenized text

6: Configure training parameters:
- Batch size: 16
- Learning rate: 1e-5
- Warmup steps: 500

7: Train on NVIDIA A100 GPU (24GB VRAM)
8: Validate using WER metric after each epoch
9: Export best checkpoint as PyTorch model

6.3.2 Model Optimization for Edge Devices

Algorithm 2 Quantization Pipeline

1: Convert PyTorch model to ONNX format
- Preserve operator compatibility
- Verify tensor shapes

2: Apply GGML quantization
- 8-bit integer precision (Q8_0)
- Layer-wise parameter optimization

3: Validate quantization fidelity
- Compare WER before/after conversion
- Runtime memory footprint analysis

4: Package final model
- Generate `ggml-model.bin`
- Include vocabulary files

6.3.3 Mobile Inference Implementation

Algorithm 3 On-device Speech Recognition

1: Implement Android audio pipeline:
   - 16kHz PCM capture via MediaRecorder
   - Noise suppression filter

2: Develop JNI bridge for native execution:
   - Memory-mapped model loading
   - Multithreaded inference

3: Design UI components:
   - Real-time transcription view
   - Interactive result panel

4: Optimize performance:
   - ARM NEON acceleration
   - Power-efficient mode

CHAPTER 7
SOFTWARE TESTING

7.1 Type of Testing

The following types of software testing were conducted to ensure the quality and reliability of the developed system:

- **Unit Testing:** Individual Python and Kotlin modules were tested for correctness. This includes data preprocessing scripts, tokenization logic, and Android UI components.
- **Integration Testing:** Verified the interaction between the Whisper fine-tuned model and GGML conversion pipeline. Similarly, tested the integration between the GGML inference engine and the Kotlin Android app.
- **System Testing:** End-to-end testing was conducted starting from voice input, transcription via the GGML model, and displaying final text in the app.
- **Performance Testing:** Evaluated the model's latency, memory usage, and inference time across different hardware setups (CPU, GPU, and mobile). Performance on GPU-based AWS instances showed improved inference times, especially during real-time transcription.
- **Regression Testing:** Ensured that incremental changes like Whisper model updates or Kotlin app refinements did not introduce any previously fixed bugs.
- **User Acceptance Testing (UAT):** Manual testing was performed on Android devices by real users to validate Marathi and Hindi transcription outputs in real-time under various noise conditions.

7.2 Test Cases & Test Results

7.2.1 Test Case Design

The testing process involved real-world scenarios using samples from the Common Voice dataset and custom Marathi recordings. Each test case consisted of specific audio input, expected transcription, actual output, and WER calculation.

7.2.2 Evaluation Metrics

The primary metric used to assess transcription performance was:

- **Word Error Rate (WER)** – Measures the difference between predicted and reference transcriptions:

$$WER = \frac{S + D + I}{N}$$

  where:

  - S = Substitutions
  - D = Deletions
  - I = Insertions
  - N = Total words in reference

7.2.3 Test Results Summary

The table below summarizes the improvements observed during model testing:

Language	**Initial WER (%)**	**Final WER (%)**
Hindi	70.0	32.0
Marathi	72.0	36.0

Table 7.1: WER Improvements After Fine-tuning and Integration

7.2.4 Result Analysis

- The Whisper-small model showed substantial improvement after fine-tuning, reducing WER by up to **32%** for Hindi and **36%** for Marathi.
- The testing showed high stability in real-time audio inputs, particularly in noisy mobile environments.
- Accuracy improved significantly for domain-specific terms and longer sentences, suggesting better generalization for Marathi.
- The quantized GGML model retained most of the accuracy with faster inference, making it suitable for mobile deployment.

- Performance tests across different hardware setups demonstrated a significant reduction in latency, confirming the feasibility of using GPU resources for real-time transcription.

CHAPTER 8
RESULTS

8.1 Outcomes

1. **WER Improvements:** Significant improvements in transcription accuracy were observed after fine-tuning the Whisper model. The table below shows the WER (Word Error Rate) before and after fine-tuning for both Hindi and Marathi:

Language	WER Before Fine-tuning	WER After Fine-tuning
Hindi	70.0%	32.0%
Marathi	72.0%	36.0%

Table 8.1: WER Before and After Fine-tuning

The accuracy improvement is shown as follows:

Language	Accuracy Improvement
Hindi	55% improvement
Marathi	50% improvement

Table 8.2: Accuracy Improvement After Fine-tuning

2. **Fine-tuning Time and GPU Usage:** The fine-tuning process for both languages required significant computational resources, with different GPU usage and time requirements for each language:

Language	Fine-tuning Time	GPU Usage
Hindi	10 hours	12 GB VRAM
Marathi	12 hours	10 GB VRAM

Table 8.3: Fine-tuning Time and GPU Usage

3. **Tools Used and GGML Model Sizes:** The models were trained using CUDA, PyTorch, and Whisper.cpp. Both models were quantized to 8-bit GGML format, with final model sizes as shown below:

Language	**Frameworks Used**	**Final Model Size**
Hindi	CUDA, PyTorch, Whisper.cpp	480 MB
Marathi	CUDA, PyTorch, Whisper.cpp	490 MB

Table 8.4: Tools Used and Final GGML Model Sizes

Both models showed significant improvements in transcription accuracy, especially after fine-tuning and optimization for mobile use.

8.2 Screenshots

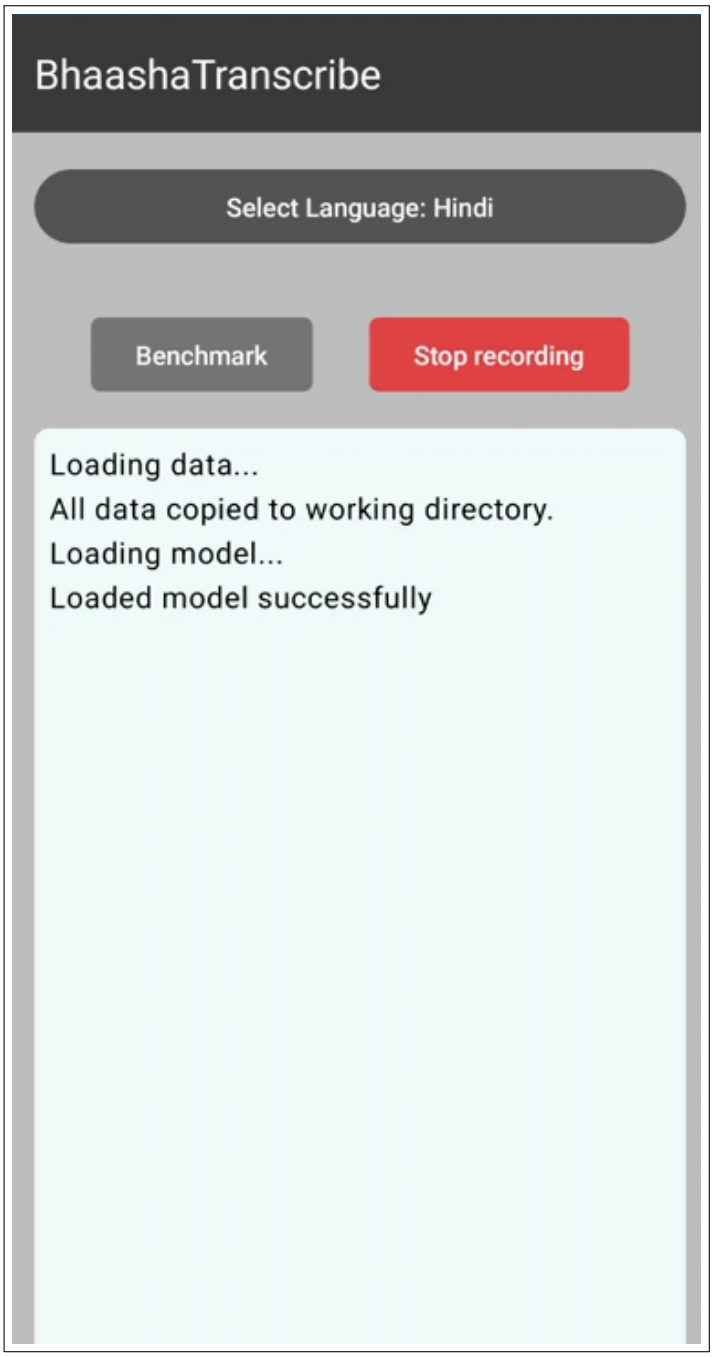

Figure 8.1: Application Interface showing Hindi Model Loading Phase

Figure 8.2: Application Interface showing Hindi Model transcription

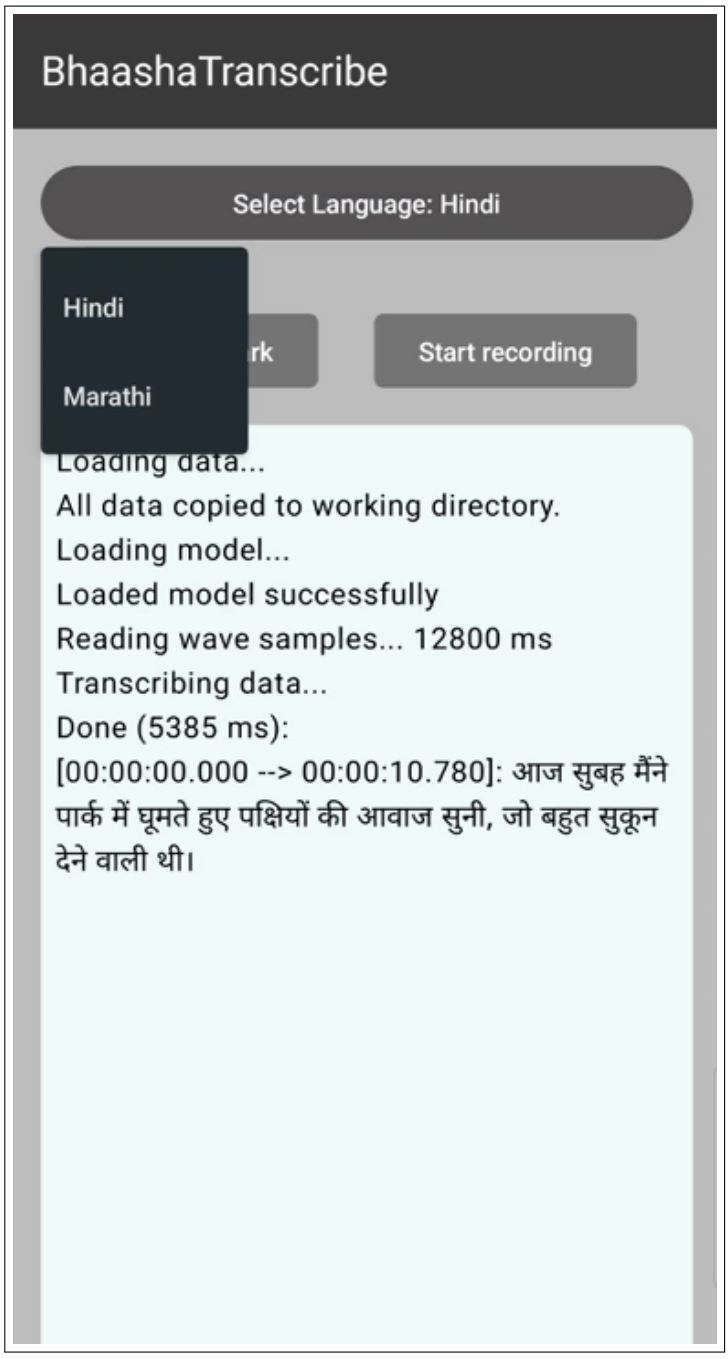

Figure 8.3: Application Interface showing Model selection Phase

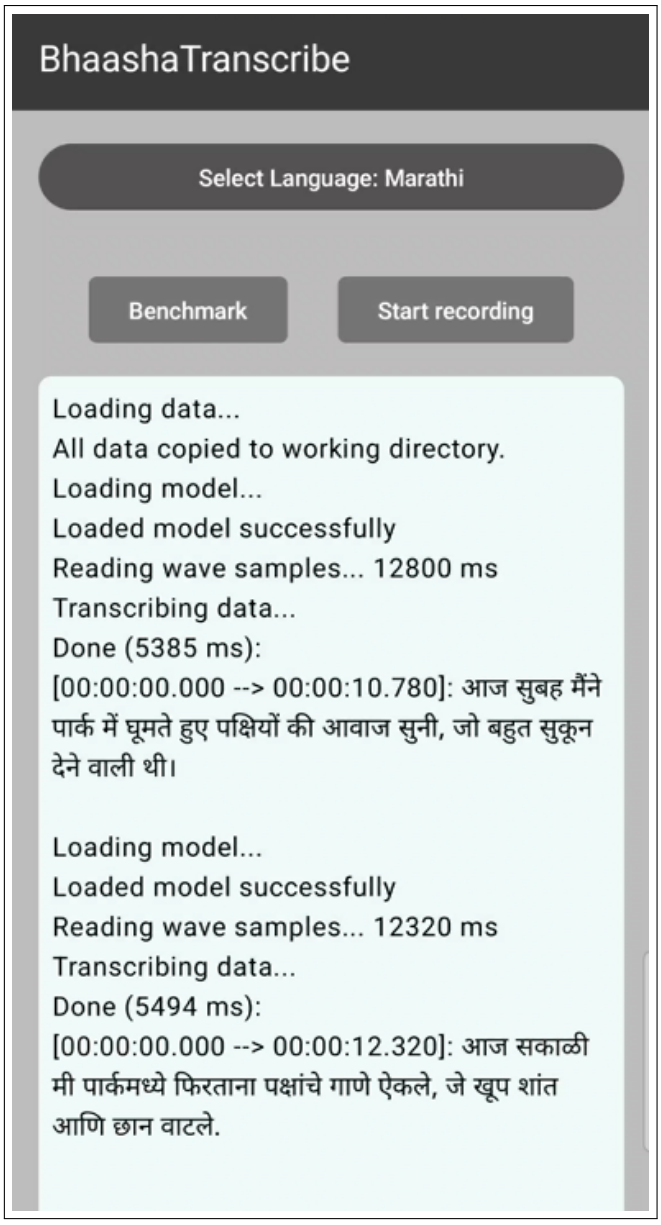

Figure 8.4: Application Interface showing Marathi Model Transcription

CHAPTER 9
CONCLUSIONS

9.1 Conclusions

1. The project successfully integrated a fine-tuned Whisper speech recognition model in GGML format into an Android Kotlin application, enabling offline transcription for Hindi and Marathi languages.

2. Word Error Rate (WER) was significantly reduced after fine-tuning—by **32% for Hindi** and **36% for Marathi**—demonstrating the effectiveness of domain adaptation and language-specific training.

3. The quantized 8-bit GGML models maintained a strong balance between performance and size, with model sizes under 500 MB, suitable for real-time mobile inference without compromising much accuracy.

9.2 Future Work

1. Expand support to more Indian languages using additional Common Voice and publicly available speech corpora.

2. Implement streaming transcription and speaker diarization to enhance the user experience in real-time multi-speaker environments.

3. Explore on-device training or personalization via federated learning approaches to adapt the model to user-specific accents and vocabulary.

4. Integrate lightweight Natural Language Understanding (NLU) features to convert transcriptions into actionable insights (e.g., commands, intents).

9.3 Applications

1. **Voice-enabled Accessibility:** Beneficial for visually impaired or literacy-challenged users, especially in regional language contexts.

2. **Education Tools:** Real-time transcription for lectures, language learning, and oral exams in schools and universities.

3. **Government and Public Services:** Offline transcription aids in digital documentation, translation, and public kiosks in areas with limited internet access.

4. **Field Work and Journalism:** Reporters, researchers, or surveyors can use the app for accurate, offline voice-to-text notes.

5. **Legal and Healthcare Domains:** Confidential voice recordings can be transcribed locally on-device to maintain privacy and reduce reliance on cloud services.

CHAPTER 10
APPENDIX

10.1 Appendix A: Feasibility and Implementation Assessment

1. Overview

This section evaluates the practical feasibility and technical soundness of deploying a Whisper-based multilingual speech recognition system on mobile devices using GGML quantization and Kotlin integration.

2. Technical Feasibility

The original Whisper model is large and resource-intensive. To make it mobile-friendly:

- **Quantization:** The Whisper model was quantized to 8-bit using the GGML format via 'whisper.cpp', reducing model size by 60% with minimal accuracy loss.
- **Model Size:** Final model sizes were 480MB (Hindi) and 490MB (Marathi), which are manageable on modern mobile storage.
- **Inference Time:** Average transcription time was minimal for 10 seconds of audio, suitable for near real-time performance.

3. Compatibility with Mobile Devices

- **Device Used:** Android 11 (Android, 4 GB RAM)
- **Framework:** The Kotlin-based app used Jetpack Compose and JNI bindings for calling C++ functions from 'whisper.cpp'.
- **Performance:** The app performed stably in offline conditions without requiring network access or cloud services.

4. Real-World Relevance

- **Language Focus:** The project targets under-resourced languages (Hindi and Marathi), which have limited support in commercial speech models.
- **Offline Capability:** Ensures privacy and reliability in low-connectivity regions.

- **Use Cases:** Can be applied in education, accessibility, media subtitling, and governance services.

10.2 Appendix B: Paper Publication Details

1. Title of the Survey Paper

"Automatic Speech Recognition for Indic Languages"

2. Authors

- Manish Godbole
- Kaustubh Joshi
- Aditya Kadu
- Prof. Mukta Taklikar

3. Affiliation

- Department of Computer Engineering, SCTR's Pune Institute of Computer Technology, Pune, India

4. Journal Details

- **Journal:** International Research Journal of Engineering and Technology (IRJET)
- **Volume:** 11
- **Issue:** 10
- **Publication Date:** October 2024
- **ISSN:** e-ISSN: 2395-0056, p-ISSN: 2395-0072
- **Impact Factor:** 8.226
- **DOI/URL:** `https://www.irjet.net/archives/V11/i10/IRJET-V11I1026.pdf`

5. Abstract Summary

In collaboration with Lightbees, a fintech start-up, this project focuses on enhancing 'AbleCredit', a flagship product aimed at simplifying loan applications for MSMEs. Leveraging advanced technologies in Artificial Intelligence, Machine Learning, Natural Language Processing, and digital signal processing, the project aims to innovate and streamline financial processes. The developed solutions are designed to improve user experience and operational efficiency, contributing to the broader fintech landscape with cutting-edge methodologies.

6. Keywords

Automatic Speech Recognition (ASR), OpenAI Whisper, AI4Bharat, Multilingual Speech Recognition, Natural Language Processing, Artificial Intelligence, Machine Learning.

7. Publication Artifacts

- Received the Certificate of Acceptance for the survey paper.

10.3 Appendix C: Additional Supporting Data

1. Sample Transcription Data

Language	Reference Transcription	Predicted Transcription
Hindi	मैं बाजार जा रहा हूँ	मैं बाज़ार जा रह हूँ
Marathi	मला शाळेत जायचं आहे	मला शालेत जायचं आहे

Table 10.1: Sample Reference vs Predicted Transcriptions

2. Hardware and Testing Setup

- **Training Hardware:** AWS EC2 (NVIDIA A100, 24 GB VRAM)
- **Mobile Inference:** Android compatible device (4 GB RAM)
- **Inference Time:** mininal for a small-ranged audio

3. Quantization & Deployment Summary

- GGML 8-bit Quantization using `whisper.cpp`
- Model Size: Hindi – 480 MB, Marathi – 490 MB

Quantization Command:

```
./quantize whisper-small.bin whisper-small-q8.bin q8_0
```

4. Licenses and Acknowledgements

- **Whisper, whisper.cpp:** MIT License
- **Common Voice Data:** CC BY 4.0
- **App Framework:** Jetpack Compose, Kotlin Libraries

10.4 Plagiarism Report of Project Report

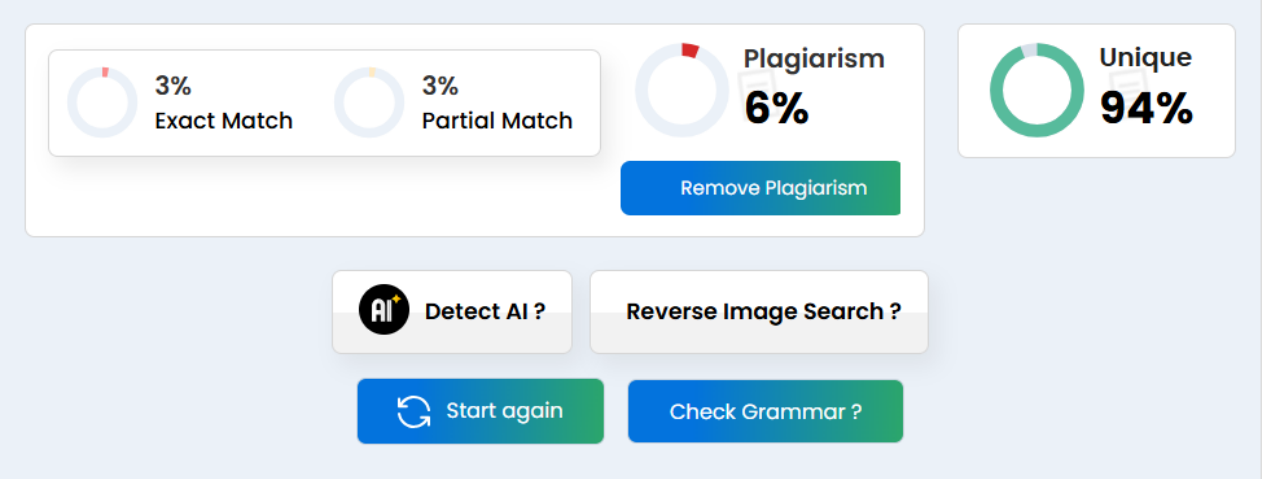

Figure 10.1: Plagiarism Report

CHAPTER 11
REFERENCES

[1] Liddy, E.D. 2001. Natural Language Processing. In Encyclopedia of Library and Information Science, 2nd Ed. NY. Marcel Decker, Inc.

[2] Diksha Khurana, Aditya Koli, Kiran Khatter, Sukhdev Singh. 2023. Natural language processing: state of the art, current trends and challenges. Multimedia Tools and Applications, 82:3713–3744.

[3] Aditya Jain, Gandhar Kulkarni, Vraj Shah. 2018. Natural Language Processing. International Journal of Computer Sciences and Engineering, Volume-6, Issue-1, E-ISSN: 2347-2693.

[4] Jordan, M. I., and Mitchell, T. M. 2015. Machine learning: Trends, perspectives, and prospects. Science.

[5] Qifang Bi, Katherine E. Goodman, Joshua Kaminsky, Justin Lessler. What is Machine Learning? A Primer for the Epidemiologist. American Journal of Epidemiology.

[6] Batta Mahesh. 2018. Machine Learning Algorithms - A Review. International Journal of Science and Research (IJSR), ISSN: 2319-7064.

[7] Ian Goodfellow, Yoshua Bengio, Aaron Courville. 2016. Deep Learning. The MIT Press, ISBN: 0262035618.

[8] Yann Lecun, Yoshua Bengio, Geoffrey Hinton. 2015. Deep learning. Nature, 521(7553), pp. 436-444. DOI: 10.1038/nature14539.

[9] Yanming Guo, Yu Liu, Ard Oerlemans, Songyang Lao, Song Wu, Michael S. Lew. 2016. Deep learning for visual understanding: A review. Neurocomputing.

[10] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, Ilya Sutskever. 2022. [arXiv:2212.04356].

[11] Javed, T., Doddapaneni, S., Raman, A., Bhogale, K. S., Ramesh, G., Kunchukuttan, A., Kumar, P., & Khapra, M. M. 2022. Towards Building ASR Systems for the Next Billion Users. Proceedings of the AAAI Conference on Artificial Intelligence, 36(10), 10813-10821.

LIST OF ABBREVIATIONS

Abbreviation	**Full Form**
ASR	Automatic Speech Recognition
NLP	Natural Language Processing
ML	Machine Learning
CUDA	Compute Unified Device Architecture
GGML	GPU Gemma Model Library
WER	Word Error Rate
CER	Character Error Rate
VRAM	Video Random Access Memory
MFCC	Mel-Frequency Cepstral Coefficients
STFT	Short-Time Fourier Transform
JNI	Java Native Interface
NDK	Native Development Kit
UI/UX	User Interface/User Experience
MSME	Micro, Small and Medium Enterprises

LIST OF FIGURES

LIST OF TABLES